

FactoryOS AI — Phase 1 Documentation

Deliverables

1. Product Requirements Document (PRD)

Purpose

Defines what FactoryOS AI is, why it exists, who will use it, and what will be built.

Contents

- Product Overview
- Vision
- Business Goals
- Target Users
- Problem Statement
- Features
- User Stories
- Success Metrics
- Roadmap

1.1 Product Overview

Field	Details
Product Name	FactoryOS AI
Product Type	Enterprise Industrial AI Platform
Target Sector	Heavy Industry — Oil & Gas, Pharma, Steel, Power

Primary Value Prop	Unified AI brain for industrial knowledge, maintenance, and compliance
MVP Target	Hackathon Demo — Working Prototype
Stack	React, FastAPI, Claude API, Neo4j, ChromaDB

1.2 Problem Being Solved

Industrial organizations operate across 7–12 disconnected document systems. Engineers spend 35% of their working hours searching for information (McKinsey 2024). An estimated 25% of India’s experienced industrial engineers are approaching retirement, and their tacit knowledge is undocumented. Fragmented documentation causes 18–22% of unplanned downtime events in Indian heavy industry.

1.3 Solution Statement

FactoryOS AI is a multi-agent, RAG-powered Industrial Operating System that ingests every document type — PDFs, P&IDs, SOPs, inspection reports, maintenance logs, emails — and makes them collectively queryable, actionable, and continuously updated. It connects knowledge across machines, people, vendors, and workflows through a live knowledge graph.

1.4 Success Metrics

Metric	Target
Document search time	Reduce by 80%
First-time fix rate	Improve by 30%
Compliance readiness time	Reduce by 70%

Unplanned downtime **Reduce by 25%**

Knowledge retention **100% — no knowledge lost on retirement**

1.5 Out of Scope (V1)

- **Hardware IoT sensor integration (simulated in MVP)**
- **ERP direct integration (file import only)**
- **Mobile native app**
- **Real-time SCADA feed**

2. Vision & Problem Statement

Vision

Every Document. Every Machine. One AI Brain.

Build an AI-powered Industrial Operating System that centralizes industrial knowledge, predicts failures, automates workflows, and preserves organizational expertise.

Problems Solved

- Information silos
- Knowledge loss
- Reactive maintenance
- Compliance complexity
- Slow troubleshooting
- Manual reporting
- Disconnected systems

2.1 Vision

Factories today are rich in data but poor in connected knowledge.

FactoryOS AI envisions a world where every industrial organization — regardless of size or digitization level — has access to an AI brain that captures expertise, connects information, explains decisions, predicts risks, and orchestrates workflows. A factory that learns from every document, every maintenance activity, every incident, and every human interaction.

2.2 Mission

To eliminate industrial knowledge silos by building the world's most comprehensive Industrial Operating System — one that transforms scattered documents into connected, actionable, explainable intelligence.

2.3 Problem Deep-Dive

Information Fragmentation

Maintenance teams use CMMS. Operations use SCADA. Safety teams use SharePoint. Engineers have local folders. Finance uses ERP. No single system sees across all of them.

The Knowledge Cliff

25% of experienced Indian industrial engineers will retire in the next 5 years. Their knowledge exists only in their heads. Once they leave, it is gone forever.

Reactive Maintenance Culture

Most Indian factories operate on breakdown maintenance. They fix equipment after it fails rather than predicting and preventing failures. This causes 18–22% of all unplanned downtime.

Compliance Burden

Preparing for a Factory Act or ISO audit requires manually collecting documents from 5–8 different departments. This takes weeks and is error-prone.

The Language of Documents

Industrial documents are messy. Handwritten maintenance logs. Scanned SOPs. P&ID diagrams. Hindi-English mixed notes. OCR-unfriendly equipment tags. No existing system handles this diversity.

2.4 Why Now

- LLMs are now capable of reasoning over long technical documents
- RAG + Knowledge Graphs make contextual industrial reasoning feasible
- Claude API enables citation-grounded, explainable AI responses
- Vector databases are production-ready and affordable
- India's Industry 4.0 push (National Manufacturing Policy) creates urgency

2.5 Unique Angle Added (My Additions)

1. **Shift Intelligence Layer** — AI-powered shift handover documentation
2. **Knowledge Decay Detection** — Automatic flagging of outdated SOPs and manuals
3. **Incident Learning Loop** — Self-improving safety intelligence
4. **Oil & Gas Vertical Focus** — OISD compliance as primary compliance framework
5. **Brownfield Onboarding Mode** — Value from Day 1 even with only scanned paper documents

3. User Personas

Define every user.

Example:

Plant Manager

Goals

- Increase production
- Reduce downtime

Pain Points

- Too many dashboards
- No centralized knowledge

Uses

- Executive Dashboard
- Risk Radar
- Cost Analysis

Persona 1 — The Field Maintenance Engineer

Name: Ravi Kumar, 34 **Role:** Maintenance Engineer, Oil Refinery **Experience:** 8 years **Pain Points:**

- Spends 2–3 hours finding the right manual during breakdowns
- Equipment history is scattered across paper logs and old CMMS entries
- Shift handover is verbal — information gets lost
- Has no way to check if a similar failure happened before

What FactoryOS Gives Him:

- Instant manual lookup by equipment tag
 - RCA agent that checks full history automatically
 - AI-generated shift handover summary
 - Similar incident retrieval with solutions
-

Persona 2 — The Plant Safety & Compliance Officer

Name: Sunita Sharma, 42 **Role:** HSE Manager **Experience:** 15 years **Pain Points:**

- Audit preparation takes 3–4 weeks manually

- Cannot track which SOPs are outdated
- No visibility into compliance gaps until an inspector finds them
- Incident reports are filed but lessons are never extracted

What FactoryOS Gives Her:

- Real-time compliance dashboard (OISD, Factory Act, ISO)
 - Automatic outdated SOP flagging (Knowledge Decay Detection)
 - AI-generated audit evidence packages
 - Incident learning loop that surfaces patterns before they become accidents
-

Persona 3 — The Plant Manager / Operations Head

Name: Rajesh Mehta, 52 **Role:** Plant Manager **Experience:** 25 years (retiring in 3 years) **Pain Points:**

- 25 years of experience is in his head, not documented
- Cannot see operational health across all departments in one view
- Relies on verbal updates from team leads
- No proactive alerts — learns about problems after they happen

What FactoryOS Gives Him:

- Executive intelligence dashboard
 - AI-captured institutional knowledge from his decisions and meetings
 - Cross-department operational visibility
 - Predictive maintenance alerts before failures occur
-

Persona 4 — The Junior Engineer / New Hire

Name: Priya Singh, 24 **Role:** Graduate Engineer Trainee **Experience:** 6 months **Pain Points:**

- No structured onboarding — learns by shadowing seniors
- Cannot find the right document without asking someone
- Does not know which SOP applies to which situation
- Makes mistakes because she cannot find relevant precedents

What FactoryOS Gives Her:

- Natural language document search — ask questions, get answers
 - Automatic SOP recommendations based on equipment tag
 - Similar incident history for any situation she encounters
 - Knowledge graph that shows how everything connects
-

Persona 5 — The Procurement / Vendor Manager

Name: Anil Verma, 38 **Role:** Procurement Manager **Experience:** 12 years **Pain Points:**

- Does not know what spare parts are needed until maintenance requests them
- Cannot track vendor performance across purchase history
- OEM recommendations are buried in manuals he cannot search

What FactoryOS Gives Him:

- Vendor and spare parts intelligence from knowledge graph
- Predictive maintenance → automatic spare part demand forecasting
- OEM-recommended spare parts extracted directly from manuals

Maintenance Engineer

Goals

- Repair equipment quickly

Uses

- Industrial GPT
- Root Cause AI
- Manuals
- Digital Twin

Repeat for:

- Reliability Engineer
- Safety Officer
- Operator
- Auditor
- Executive

Output: ~10 pages

4. Functional Requirements

Everything the software must do.

Example

Document Module

- Upload PDF
- OCR
- Metadata Extraction
- Version Control

Search Module

- Semantic Search
- Hybrid Search
- Graph Search

Digital Twin

- Display assets
- Show manuals
- Show history
- Live status

Continue for every module.

Module 1 — Document Intelligence

ID	Requirement	Priority
FR-01	Accept PDF, Word, Excel, scanned images, P&ID drawings	P0
FR-02	Perform OCR on scanned documents	P0
FR-03	Auto-classify documents by type (Manual, SOP, Inspection, Incident)	P0
FR-04	Extract equipment tags, dates, personnel names, parameters	P0
FR-05	Chunk documents semantically and generate embeddings	P0
FR-06	Link documents to equipment in knowledge graph	P1
FR-07	Detect and flag documents older than configurable threshold	P1

Module 2 — Industrial GPT (Query Interface)

ID	Requirement	Priority
FR-08	Accept natural language queries	P0
FR-09	Return answers with source citations and confidence scores	P0
FR-10	Compare two documents on request	P1
FR-11	Generate inspection reports from template + data	P1

FR-12	Support follow-up questions (multi-turn conversation)	P1
-------	---	----

FR-13	Return related documents alongside primary answer	P1
-------	---	----

Module 3 — Knowledge Graph

ID	Requirement	Priority
FR-14	Auto-build graph nodes for Equipment, Document, Person, Vendor, Incident	P0
FR-15	Auto-build edges (has_manual, maintained_by, caused_incident, etc.)	P0
FR-16	Visualize graph in UI	P1
FR-17	Support graph traversal queries	P1

Module 4 — Root Cause Analysis (RCA)

ID	Requirement	Priority
FR-18	Accept failure description as input	P0
FR-19	Gather maintenance history, manuals, sensor data, incidents automatically	P0
FR-20	Output timeline → evidence → probability → conclusion → recommendation	P0

FR-21	Cite every piece of evidence used	P0
-------	-----------------------------------	----

Module 5 — Predictive Maintenance

ID	Requirement	Priority
FR-22	Accept sensor data (CSV or simulated)	P1
FR-23	Output equipment health score (0–100)	P1
FR-24	Predict Remaining Useful Life (RUL)	P1
FR-25	Generate maintenance schedule recommendations	P1

Module 6 — Compliance Intelligence

ID	Requirement	Priority
FR-26	Map factory documents against OISD / ISO / Factory Act requirements	P0
FR-27	Identify missing inspections and expired certificates	P0
FR-28	Generate audit evidence package automatically	P1
FR-29	Show compliance gap dashboard	P0

Module 7 — Shift Intelligence (NEW)

ID	Requirement	Priority
FR-30	Accept outgoing shift notes (text or voice)	P0
FR-31	AI-parse and structure into standard handover format	P0
FR-32	Generate incoming shift briefing document	P0
FR-33	Flag critical items requiring immediate attention	P0

Module 8 — Incident Learning Loop (NEW)

ID	Requirement	Priority
FR-34	Accept incident reports as input	P0
FR-35	Extract lessons learned automatically	P0
FR-36	Check if similar incident occurred before	P0
FR-37	Alert sibling equipment if pattern detected	P1
FR-38	Auto-suggest SOP updates based on incident patterns	P1

Module 9 — Knowledge Decay Detection (NEW)

ID	Requirement	Priority
FR-39	Track document creation and last-modified dates	P0
FR-40	Flag SOPs older than configurable threshold (default: 2 years)	P0
FR-41	Flag manuals for decommissioned or superseded equipment	P1
FR-42	Generate "Knowledge Health Report"	P1

5. Non-Functional Requirements

Performance

Security

Scalability

Availability

Reliability

Accessibility

Latency

Example

Chat response

< 5 sec

Search

< 1 sec

99.9% uptime

Role-based access

Audit Logs

Encryption

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance

Requirement	Target
Query response time (RAG)	< 5 seconds

Document ingestion time (10MB PDF) < 30
seconds

Knowledge graph query < 2 seconds

Concurrent users supported (MVP) 50

Concurrent users supported
(Production) 10,000+

5.2 Reliability

- System uptime: 99.9% (production)
- Document ingestion pipeline: idempotent (safe to retry)
- All AI responses must include fallback if model unavailable
- Offline mode for critical document access (read-only)

5.3 Security

- Role-based access control (RBAC)
- All data encrypted at rest (AES-256) and in transit (TLS 1.3)
- Document-level access permissions
- Audit trail for every AI query and response
- No PII stored in vector embeddings
- On-premise deployment option for sensitive industries

5.4 Scalability

- Horizontal scaling of document ingestion workers
- Vector database sharding by plant/site
- Knowledge graph partitioned by organizational unit
- Multi-tenant architecture (each factory = isolated namespace)

5.5 Explainability

- Every AI answer must cite source documents
- Confidence score required on all recommendations
- RCA outputs must include evidence chain
- No black-box decisions — every recommendation must be traceable

5.6 Accessibility

- Web-first (desktop optimized)
 - Mobile responsive for field engineers
 - Hindi language support in query interface
 - High-contrast mode for factory floor environments
-
-

6. Core AI Use Cases

Document Intelligence

Industrial GPT

Predictive Maintenance

Compliance AI

Root Cause Investigation

Digital Twin

Workflow Automation

Executive AI

Voice Copilot

Vision AI

Each use case should include:

Objective

Input

AI Process

Output

Business Value

6. CORE AI USE CASES

UC-01 — "Why did Pump-21 fail?"

Trigger: Engineer types question in Industrial GPT **Flow:** Query → Search Agent → Maintenance history + Manual + Incident logs → RCA Agent → Evidence timeline → Conclusion + Recommendation

Output: Structured RCA report with citations

UC-02 — "Are we audit-ready for OISD inspection next month?"

Trigger: Safety manager opens Compliance Dashboard **Flow:** Compliance Agent → Map all equipment against OISD checklist → Identify gaps → Generate evidence package **Output:** Gap report + Auto-generated audit package

UC-03 — "Generate shift handover for Night Shift B"

Trigger: Outgoing engineer submits shift notes **Flow:** Shift Intelligence Agent → Parse notes → Structure into format → Flag critical items → Generate incoming brief **Output:** Structured shift handover document

UC-04 — "Show all documents related to Boiler-12"

Trigger: Engineer searches by equipment tag **Flow:** Knowledge Graph Agent → Traverse all nodes connected to Boiler-12 → Return manuals, maintenance records, inspections, incidents **Output:** Unified document view for Boiler-12

UC-05 — "Flag all SOPs that haven't been reviewed in 2 years"

Trigger: Scheduled daily job or manual trigger **Flow:** Knowledge Decay Agent → Query document metadata → Filter by date → Generate decay report **Output:** List of stale documents with recommended actions

UC-06 — "What lessons can we learn from last month's compressor failure?"

Trigger: Incident report uploaded **Flow:** Incident Learning Agent → Extract failure mode → Search similar incidents → Identify pattern → Check sibling equipment → Suggest SOP update **Output:** Lessons learned document + alert to sibling equipment owners

UC-07 — "Predict health of Compressor-3 for next 30 days"

Trigger: Predictive Maintenance Agent (scheduled or manual) **Flow:** Sensor data + maintenance history → Predictive model → Health score + RUL + maintenance schedule **Output:** Equipment health forecast dashboard

UC-08 — "Upload this P&ID and extract all equipment tags"

Trigger: Engineer uploads engineering drawing **Flow:** Vision AI Agent + OCR → Extract equipment tags, pipe specifications, instrument tags → Add to knowledge graph **Output:** Structured equipment list + graph nodes created

UC-09 — "Compare SOP-12A with SOP-12B and tell me what changed"

Trigger: User uploads two documents **Flow:** Document Intelligence Agent → Semantic diff → Highlight changes → Flag safety-critical differences **Output:** Side-by-side comparison with change summary

UC-10 — "What spare parts do I need for quarterly Pump-21 maintenance?"

Trigger: Maintenance engineer query **Flow:** Maintenance Agent → OEM manual extraction + maintenance history + vendor graph → Generate BOM **Output:** Bill of materials with vendor and last-order information

7. AI Agent Specifications

Document every AI agent.

Example

Document Agent

Input

PDF

Output

Structured document

Responsibilities

OCR

Metadata

Chunking

Embedding

Tools

Docling

PaddleOCR

Chroma

Repeat for:

- Search Agent
- Maintenance Agent
- Compliance Agent
- Vision Agent
- Report Agent
- Executive Agent
- Workflow Agent
- Memory Agent
- Supply Chain Agent
- Digital Twin Agent

Architecture Overview

FactoryOS uses a **Multi-Agent Orchestration** architecture. An **Orchestrator Agent** receives every user request, determines which specialized agents to invoke, coordinates their execution, and synthesizes the final response.

All agents use **Claude API (claude-sonnet-4-6)** as their reasoning engine with specialized system prompts, tools, and context windows.

Agent 1 — Orchestrator Agent

Role: Master coordinator. Routes queries to specialized agents. **Input:** Raw user query **Output:** Final synthesized response **System Prompt Focus:** Task decomposition, agent selection, response synthesis **Tools:** All agent APIs **Decision Logic:**

- Keyword/intent detection → route to correct agent(s)
 - Multi-agent tasks → fan out and aggregate
 - Conflict resolution when agents return contradictory answers
-

Agent 2 — Document Intelligence Agent

Role: Ingests, processes, and structures any industrial document **Input:** PDF, Word, Excel, Image, Scanned document **Output:** Structured metadata + chunked embeddings + knowledge graph nodes **System Prompt Focus:** Industrial entity extraction (equipment tags, dates, parameters, personnel) **Tools:** OCR (PaddleOCR), PDF parser, chunker, embedding generator, graph writer **Special Handling:** P&ID diagrams, handwritten maintenance logs, multi-language documents

Agent 3 — Search & Retrieval Agent

Role: Hybrid semantic + keyword search across all knowledge sources **Input:** Natural language query **Output:** Ranked list of relevant document chunks with source metadata **System Prompt Focus:** Query expansion, re-ranking, citation formatting **Tools:** ChromaDB vector search, Elasticsearch keyword search, metadata filters **Search Strategy:** Dense retrieval → re-rank → filter by equipment/date/type → return top-k

Agent 4 — Industrial GPT Agent

Role: Answers natural language questions about industrial operations **Input:** User query + retrieved context from Search Agent **Output:** Answer with citations, confidence score, related documents **System Prompt Focus:** Industrial domain reasoning, citation grounding, uncertainty acknowledgment **Tools:**

Search Agent, Knowledge Graph Agent **Constraint:** Must always cite source. Must output confidence score. Must not hallucinate.

Agent 5 — Root Cause Analysis (RCA) Agent

Role: Investigates equipment failures systematically **Input:** Failure description + equipment tag **Output:** RCA report (Timeline → Evidence → Probability → Conclusion → Recommendation) **System Prompt Focus:** Systematic investigation, fault tree reasoning, maintenance pattern recognition **Tools:** Maintenance history lookup, manual search, incident retrieval, sensor data reader **Output Format:**

FAILURE: [description]

TIMELINE: [chronological evidence]

PROBABLE CAUSES: [ranked by probability]

SUPPORTING EVIDENCE: [citations]

RECOMMENDED ACTION: [specific steps]

PREVENTIVE MEASURES: [future prevention]

Agent 6 — Predictive Maintenance Agent

Role: Forecasts equipment health and remaining useful life **Input:** Sensor data (CSV) + maintenance history + equipment specifications **Output:** Health score, RUL prediction, maintenance schedule **System Prompt Focus:** Anomaly detection, trend analysis, maintenance optimization **Tools:** Statistical analysis, pattern matching against failure history **Models:** Time-series anomaly detection + Claude reasoning for interpretation

Agent 7 — Compliance Intelligence Agent

Role: Maps factory state against regulatory requirements **Input:** All factory documents + equipment states + inspection records **Output:** Compliance gap report + audit evidence package **System Prompt Focus:** OISD guidelines, Factory Act sections, ISO 9001/14001/45001, GMP **Tools:** Regulatory knowledge base, document metadata queries, gap analysis templates **Deliverables:** Compliance score, gap list, evidence package, recommended actions

Agent 8 — Knowledge Graph Agent

Role: Builds, queries, and traverses the industrial knowledge graph **Input:** Entities and relationships from all document agents **Output:** Graph nodes, edges, traversal results, visual data **System Prompt Focus:** Entity relationship reasoning, graph path finding **Tools:** Neo4j Cypher query generator, graph schema enforcer **Key Relationships Managed:**

- Equipment → has_manual → Document
 - Equipment → had_incident → Incident
 - Incident → led_to_action → WorkOrder
 - Engineer → maintains → Equipment
 - Vendor → supplies → SparePart
-

Agent 9 — Shift Intelligence Agent (NEW)

Role: Transforms unstructured shift notes into structured handover documents **Input:** Outgoing engineer's notes (text/voice transcript) **Output:** Structured shift handover report **System Prompt Focus:** Information extraction, priority classification, operational context **Output Sections:**

EQUIPMENT STATUS: [list with status]

PENDING ACTIONS: [priority ordered]

CRITICAL ALERTS: [immediate attention items]

COMPLETED WORK: [what was done]

NOTES FOR INCOMING SHIFT: [context and warnings]

Agent 10 — Incident Learning Agent (NEW)

Role: Extracts lessons from incidents and prevents recurrence **Input:** Incident report document **Output:** Lessons learned + similar incident history + SOP update suggestions + sibling equipment alerts **System Prompt Focus:** Failure mode analysis, pattern recognition, preventive action generation **Tools:** Incident history search, equipment relationship graph, SOP lookup **Unique Feature:** Automatically checks if same failure mode occurred in last 2 years anywhere in the plant

Agent 11 — Knowledge Decay Agent (NEW)

Role: Monitors document freshness and flags outdated knowledge **Input:** Document metadata database **Output:** Decay report listing stale documents with risk assessment **System Prompt Focus:** Industrial document lifecycle, regulatory review frequencies **Rules:**

- SOPs: Flag if not reviewed in 24 months

- Maintenance manuals: Flag if equipment model superseded
 - Inspection records: Flag if next scheduled inspection overdue
 - Certificates: Flag 30 days before expiry
-

Agent 12 — Vision AI Agent

Role: Analyzes equipment photos for defects and safety issues **Input:** Image (JPG/PNG) + equipment context **Output:** Defect report with bounding boxes and severity assessment **System Prompt Focus:** Industrial defect classification, safety compliance **Tools:** Florence-2 / YOLO for detection, Claude for interpretation **Detects:** Cracks, corrosion, leaks, loose bolts, missing PPE, damaged components

Agent 13 — Workflow Automation Agent

Role: Converts AI insights into actionable workflow triggers **Input:** Alert or recommendation from any agent **Output:** Work order, notification, calendar entry, report **System Prompt Focus:** Workflow logic, priority assignment, escalation rules **Trigger → Action Examples:**

- RCA complete → Create work order in CMMS
 - Compliance gap found → Notify safety officer
 - Predictive alert → Schedule maintenance window
-

Agent 14 — Report Generator Agent

Role: Generates formal industrial reports from AI analysis **Input:** Structured data from any agent **Output:** Formatted PDF/Word report with sections, tables, charts **Report Types:** RCA Report, Inspection Report, Shift Handover, Compliance Report, Equipment Health Report **System Prompt Focus:** Industrial report formatting, regulatory language, professional documentation

Agent 15 — Memory Agent

Role: Maintains organizational memory across all interactions **Input:** Every conversation, decision, note, meeting transcript **Output:** Searchable institutional memory store **System Prompt Focus:** Information distillation, context preservation, retrieval optimization **Special:** Captures senior engineer decisions and reasoning before retirement

Agent 16 — Executive Intelligence Agent

Role: Synthesizes operational intelligence for management dashboards **Input:** Data from all other agents **Output:** Executive summary, KPI dashboard, risk alerts, trend analysis **System Prompt Focus:** Business impact quantification, priority ranking, strategic insights **Audience:** Plant managers, operations heads, C-suite

Agent 17 — Supply Chain & Vendor Agent

Role: Connects maintenance needs with procurement intelligence **Input:** Maintenance requirements + vendor database + purchase history **Output:** Spare part recommendations with vendor details and lead times **System Prompt Focus:** Procurement optimization, vendor risk assessment **Tools:** Vendor graph traversal, OEM manual extraction, purchase history analysis

8. System Architecture

High-level system.

Users



Next.js



FastAPI



LangGraph



AI Agents



Knowledge Layer



Databases

Include architecture diagrams.

8.1 Architecture Principles

- 1. **Agent-First:** Every capability implemented as an autonomous agent
- 2. **RAG-Native:** All responses grounded in retrieved evidence
- 3. **Graph-Connected:** All entities linked through knowledge graph
- 4. **Explainable by Design:** No black-box outputs
- 5. **Modular:** Each module independently deployable
- 6. **Brownfield-Compatible:** Works with scanned paper documents from Day 1

8.2 System Components



Claude API (claude-sonnet-4-6) | LangGraph

Tool Calling | Multi-turn Memory



VECTOR DB | GRAPH DB | RELATIONAL DB

ChromaDB | Neo4j | PostgreSQL

Embeddings | Knowledge | Users, Docs, Logs

Graph



STORAGE LAYER

MinIO (Documents) | Redis (Cache)



9. High-Level Design (HLD)

- Shows
 - Subsystems
 - Responsibilities
 - Communication
 - Technologies
 - Components
- Example
 - Frontend
 - ↓
 - API Gateway
 - ↓
 - AI Layer
 - ↓
 - Data Layer
 - ↓
 - Storage



And

ne

Document Upload



File Type Detection



OCR (if scanned) → PaddleOCR

PDF Extract → PyMuPDF

Image → Florence-2

Tabular → pandas



Document Classification (Claude)



Entity Extraction

(Equipment Tags, Dates, Personnel, Parameters)



Semantic Chunking (512 tokens, 50 overlap)



Embedding Generation (text-embedding-3-small)



ChromaDB (vectors)

PostgreSQL (meta)

Neo4j (entities)

MinIO (raw file)



User Query



Intent Classification



Simple Q&A → GPT Agent

RCA Request → RCA Agent

Compliance → Compliance Agent

Shift → Shift Agent

Incident → Incident Agent



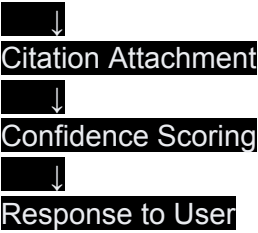
Context Retrieval (Hybrid RAG)



Agent Reasoning (Claude API)



Response Synthesis



9.3 Technology Stack

9.3 Technology Stack

Layer	Technology	Reason
Frontend	React + TypeScript + Tailwind	Fast, type-safe, production-ready
UI Components	shadcn/ui	Industrial-grade component library
Backend API	FastAPI	Async, fast, Python ecosystem
Task Queue	Celery + Redis	Async document ingestion
AI Reasoning	Claude API (claude-sonnet-4-6)	Citation-grounded, safe, explainable
Agent Framework	LangGraph	Stateful multi-agent orchestration
Vector Store	ChromaDB	Open-source, local-first

Graph Database	Neo4j	Industry standard knowledge graphs
Relational DB	PostgreSQL	ACID compliant structured data
Object Storage	MinIO	S3-compatible, on-premise
Cache	Redis	Fast caching for frequent queries
OCR	PaddleOCR	Multilingual, handles handwriting
Vision	Florence-2 / YOLO	Equipment defect detection
Deployment	Docker + Docker Compose	Portable, reproducible

10. Low-Level Design (LLD)

Detailed module design.

Example

Document Upload



OCR



Parser



Chunker



Embedding



Vector Store



Knowledge Graph



Retriever



LLM

Include sequence diagrams.

And

10. LOW-LEVEL DESIGN (LLD)

10.1 Document Ingestion Worker

python

```
class DocumentIngestionWorker:
```

```
    def ingest(self, file_path: str, metadata: dict) -> IngestionResult:
```

```
        # Step 1: Detect file type
```

```
        file_type = self.detect_type(file_path)
```

```
        # Step 2: Extract raw text
```

```
        raw_text = self.extract_text(file_path, file_type)
```

```
        # Step 3: OCR if needed
```

```
        if self.needs_ocr(raw_text):
```

```
            raw_text = self.ocr_engine.process(file_path)
```

```
        # Step 4: Classify document
```

```
        doc_type = self.classifier.classify(raw_text[:2000])
```

```
        # Step 5: Extract entities
```

```
        entities = self.entity_extractor.extract(raw_text)
```

```
        # Returns: {equipment_tags, dates, personnel, parameters}
```

```
        # Step 6: Chunk text
```

```

chunks = self.chunker.chunk(
    raw_text,
    chunk_size=512,
    overlap=50,
    strategy="semantic"
)

# Step 7: Generate embeddings
embeddings = self.embedder.embed(chunks)

# Step 8: Store everywhere
doc_id = self.store_document(file_path, metadata, doc_type)
self.vector_store.upsert(doc_id, chunks, embeddings)
self.graph.create_document_node(doc_id, entities)

return IngestionResult(doc_id=doc_id, status="success")

```

10.2 RCA Agent Logic

python

class RCAAgent:

```

def investigate(self, failure_description: str, equipment_tag: str) -> RCAReport:

    # Gather evidence in parallel
    evidence = {
        "maintenance_history": self.get_maintenance_history(equipment_tag),
        "relevant_manuals": self.search_manuals(failure_description, equipment_tag),
    }

```

```
"similar_incidents": self.find_similar_incidents(failure_description),  
"sensor_data": self.get_sensor_anomalies(equipment_tag, days=30),  
"operator_notes": self.search_operator_notes(equipment_tag)  
}
```

```
# Build context for Claude
```

```
context = self.format_evidence(evidence)
```

```
# Claude reasons over evidence
```

```
response = claude_api.call(  
    system_prompt=RCA_SYSTEM_PROMPT,  
    user_message=f"""  
Equipment: {equipment_tag}  
Failure: {failure_description}  
Evidence: {context}
```

```
Perform systematic RCA. Output:
```

1. Timeline of events
2. Probable causes (ranked by probability)
3. Supporting evidence for each cause
4. Recommended corrective actions
5. Preventive measures

```
""",
```

```
tools=[citation_tool, confidence_score_tool]
```

```
)
```

```
return RCAReport(  
    equipment=equipment_tag,  
    failure=failure_description,  
    timeline=response.timeline,  
    causes=response.probable_causes,  
    evidence=response.citations,  
    recommendations=response.actions,  
    confidence=response.confidence_score  
)
```

10.3 Hybrid RAG Search

python

```
class HybridSearchEngine:
```

```
    def search(self, query: str, filters: dict = None, top_k: int = 10):
```

```
        # Dense search (semantic)
```

```
        query_embedding = self.embedder.embed(query)
```

```
        dense_results = self.vector_store.search(  
            embedding=query_embedding,  
            top_k=top_k * 2,  
            filters=filters  
)
```

```
        # Sparse search (keyword BM25)
```

```
        sparse_results = self.bm25_index.search(  
            query=query,
```

```
        top_k=top_k * 2,  
        filters=filters  
    )  
  
    # Reciprocal Rank Fusion  
    fused_results = self.rrf_merge(dense_results, sparse_results)  
  
    # Re-rank with cross-encoder  
    reranked = self.reranker.rerank(query, fused_results[:top_k * 2])  
  
    # Attach source metadata  
    results_with_citations = self.attach_citations(reranked[:top_k])  
  
    return results_with_citations
```

11. Database Schema

Design

Tables

Relationships

Indexes

Entities

Example

Users

Machines

Documents

Maintenance

Incidents

Reports

Work Orders

Sensors

Vendors

PostgreSQL Tables

sql

-- Users and access control

CREATE TABLE users (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

name VARCHAR(255) NOT NULL,

email VARCHAR(255) UNIQUE NOT NULL,

role VARCHAR(50) NOT NULL, -- admin, engineer, safety_officer, manager

plant_id UUID REFERENCES plants(id),

```
department VARCHAR(100),  
  
created_at TIMESTAMP DEFAULT NOW(),  
  
last_active TIMESTAMP  
  
);
```

-- Plants / Factory Sites

```
CREATE TABLE plants (  
  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    name VARCHAR(255) NOT NULL,  
  
    location VARCHAR(255),  
  
    industry_type VARCHAR(100), -- oil_gas, pharma, steel, power  
  
    compliance_framework VARCHAR(100), -- OISD, GMP, IS  
  
    created_at TIMESTAMP DEFAULT NOW()  
  
);
```

-- Equipment Registry

```
CREATE TABLE equipment (  
  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    plant_id UUID REFERENCES plants(id),  
  
    tag VARCHAR(100) UNIQUE NOT NULL, -- e.g., PUMP-21, BOILER-12  
  
    name VARCHAR(255) NOT NULL,  
  
    equipment_type VARCHAR(100), -- pump, boiler, compressor, valve  
  
    manufacturer VARCHAR(255),  
  
    model VARCHAR(255),  
  
    installation_date DATE,  
  
    last_maintenance DATE,
```

```
health_score INTEGER DEFAULT 100,  
  
status VARCHAR(50) DEFAULT 'operational', -- operational, maintenance, failed  
  
created_at TIMESTAMP DEFAULT NOW()  
  
);
```

-- Document Registry

```
CREATE TABLE documents (  
  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    plant_id UUID REFERENCES plants(id),  
  
    filename VARCHAR(500) NOT NULL,  
  
    doc_type VARCHAR(100) NOT NULL, -- manual, sop, inspection, incident, maintenance_log  
  
    upload_date TIMESTAMP DEFAULT NOW(),  
  
    last_reviewed DATE,  
  
    review_frequency_months INTEGER DEFAULT 24,  
  
    is_stale BOOLEAN DEFAULT FALSE,  
  
    equipment_tag VARCHAR(100), -- linked equipment  
  
    uploaded_by UUID REFERENCES users(id),  
  
    storage_path VARCHAR(1000),  
  
    file_size_bytes BIGINT,  
  
    page_count INTEGER,  
  
    processing_status VARCHAR(50) DEFAULT 'pending', -- pending, processing, complete, failed  
  
    metadata JSONB  
  
);
```

-- Document Chunks (for RAG)

```
CREATE TABLE document_chunks (  

```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
document_id UUID REFERENCES documents(id),
chunk_index INTEGER NOT NULL,
chunk_text TEXT NOT NULL,
start_char INTEGER,
end_char INTEGER,
embedding_id VARCHAR(255), -- ChromaDB reference
entities_extracted JSONB
);

-- Maintenance Records

CREATE TABLE maintenance_records (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
equipment_id UUID REFERENCES equipment(id),
maintenance_type VARCHAR(100), -- preventive, corrective, predictive
performed_by UUID REFERENCES users(id),
performed_date TIMESTAMP NOT NULL,
duration_hours DECIMAL(6,2),
work_performed TEXT,
parts_replaced JSONB,
observations TEXT,
next_maintenance_due DATE,
work_order_number VARCHAR(100),
cost DECIMAL(12,2)
);
```

-- Incidents

```
CREATE TABLE incidents (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    plant_id UUID REFERENCES plants(id),  
    equipment_id UUID REFERENCES equipment(id),  
    occurred_at TIMESTAMP NOT NULL,  
    reported_by UUID REFERENCES users(id),  
    severity VARCHAR(50), -- critical, major, minor, near_miss  
    incident_type VARCHAR(100),  
    description TEXT,  
    root_cause TEXT,  
    corrective_actions TEXT,  
    lessons_learned TEXT,  
    similar_incidents_count INTEGER DEFAULT 0,  
    status VARCHAR(50) DEFAULT 'open', -- open, investigating, closed  
    document_id UUID REFERENCES documents(id)  
);
```

-- Shift Handovers

```
CREATE TABLE shift_handovers (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    plant_id UUID REFERENCES plants(id),  
    shift_date DATE NOT NULL,  
    shift_type VARCHAR(50), -- morning, afternoon, night  
    outgoing_engineer UUID REFERENCES users(id),  
    incoming_engineer UUID REFERENCES users(id),
```

```
raw_notes TEXT,  
  
ai_structured_handover JSONB, -- AI-processed structured handover  
  
critical_alerts JSONB,  
  
created_at TIMESTAMPTZ DEFAULT NOW()  
  
);
```

-- Compliance Checklist

```
CREATE TABLE compliance_items (  
  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    plant_id UUID REFERENCES plants(id),  
  
    framework VARCHAR(100), -- OISD, ISO9001, FactoryAct, GMP  
  
    requirement_code VARCHAR(100),  
  
    requirement_description TEXT,  
  
    equipment_id UUID REFERENCES equipment(id),  
  
    status VARCHAR(50), -- compliant, non_compliant, due_soon, overdue  
  
    last_checked DATE,  
  
    next_due DATE,  
  
    evidence_document_id UUID REFERENCES documents(id),  
  
    notes TEXT  
  
);
```

-- AI Query Audit Log

```
CREATE TABLE query_audit_log (  
  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    user_id UUID REFERENCES users(id),  
  
    query_text TEXT NOT NULL,
```

```
agent_used VARCHAR(100),  
response_summary TEXT,  
citations JSONB,  
confidence_score DECIMAL(4,3),  
response_time_ms INTEGER,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

12. Knowledge Graph Schema

Nodes

Machine

Manual

Vendor

Engineer

Sensor

Incident

Maintenance

Inspection

Edges

HAS_MANUAL

CONNECTED_TO

FAILED_IN

MAINTAINED_BY

Neo4j Node Types

cypher

// Equipment Node

CREATE (e:Equipment {

id: "uuid",

tag: "PUMP-21",

name: "Crude Transfer Pump",

type: "centrifugal_pump",

```
    manufacturer: "Kirloskar",  
    health_score: 78,  
    status: "operational",  
    plant_id: "uuid"  
  })
```

// Document Node

```
CREATE (d:Document {  
  id: "uuid",  
  filename: "PUMP-21-OEM-Manual-v3.pdf",  
  doc_type: "manual",  
  upload_date: "2024-01-15",  
  last_reviewed: "2024-01-15",  
  is_stale: false  
})
```

// Incident Node

```
CREATE (i:Incident {  
  id: "uuid",  
  occurred_at: "2025-06-15",  
  severity: "major",  
  failure_mode: "bearing_failure",  
  root_cause: "lubrication_failure",  
  resolved: true  
})
```

// Engineer Node

```
CREATE (eng:Engineer {  
    id: "uuid",  
    name: "Ravi Kumar",  
    role: "maintenance_engineer",  
    specialization: "rotating_equipment",  
    years_experience: 8  
})
```

// Vendor Node

```
CREATE (v:Vendor {  
    id: "uuid",  
    name: "SKF India",  
    type: "parts_supplier",  
    rating: 4.5,  
    lead_time_days: 7  
})
```

// SOP Node

```
CREATE (s:SOP {  
    id: "uuid",  
    sop_code: "MAINT-PM-PUMP-001",  
    title: "Pump Preventive Maintenance SOP",  
    version: "2.1",  
    last_reviewed: "2023-06-01",  
    is_stale: true  
})
```

```

}))

// Spare Part Node

CREATE (sp:SparePart {
    id: "uuid",
    part_number: "SKF-6205-2RS",
    description: "Deep Groove Ball Bearing",
    unit: "piece",
    current_stock: 12
})

```

Neo4j Relationship Types

cypher

```

// Equipment ← Document relationships

(e:Equipment)-[:HAS_MANUAL]->(d:Document)

(e:Equipment)-[:HAS_INSPECTION_REPORT]->(d:Document)

(e:Equipment)-[:HAS_MAINTENANCE_LOG]->(d:Document)


// Equipment ← Incident relationships

(e:Equipment)-[:HAD_INCIDENT]->(i:Incident)

(i:Incident)-[:SIMILAR_TO]->(i2:Incident)


// Equipment ← Engineer relationships

(eng:Engineer)-[:MAINTAINS]->(e:Equipment)

(eng:Engineer)-[:INVESTIGATED]->(i:Incident)

```

```
// Equipment ← SparePart relationships
```

```
(e:Equipment)-[:REQUIRES_PART]->(sp:SparePart)
```

```
(sp:SparePart)-[:SUPPLIED_BY]->(v:Vendor)
```

```
// SOP relationships
```

```
(e:Equipment)-[:GOVERNED_BY]->(s:SOP)
```

```
(i:Incident)-[:TRIGGERED_UPDATE_OF]->(s:SOP)
```

```
// Sibling equipment (same type, same plant)
```

```
(e1:Equipment)-[:SIBLING_OF]->(e2:Equipment)
```

```
// Compliance
```

```
(e:Equipment)-[:REQUIRES_COMPLIANCE]->(c:ComplianceItem)
```

Key Graph Queries

cypher

```
// All documents linked to PUMP-21
```

```
MATCH (e:Equipment {tag: "PUMP-21"})-[r]-(d:Document)
```

```
RETURN e, r, d
```

```
// Similar incidents to a given incident
```

```
MATCH (i:Incident {id: "uuid"})-[:SIMILAR_TO]-(similar:Incident)
```

```
RETURN similar ORDER BY similar.occurred_at DESC LIMIT 5
```

```
// Full context for a failure investigation
```

```
MATCH path = (e:Equipment {tag: "PUMP-21"})-[*1..3]-(connected)
```

WHERE connected:Document OR connected:Incident OR connected:Engineer

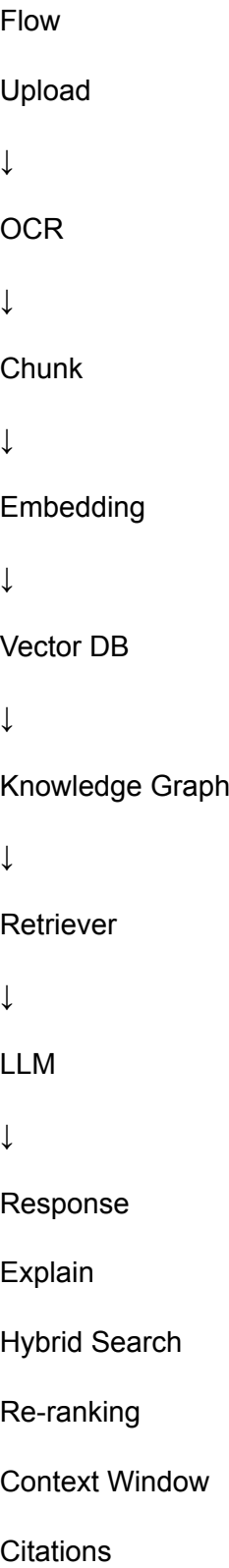
RETURN path LIMIT 50

// Stale SOPs for active equipment

MATCH (e:Equipment {status: "operational"})-[:GOVERNED_BY]->(s:SOP {is_stale: true})

RETURN e.tag, s.sop_code, s.last_reviewed

13. RAG & GraphRAG Flow



13. RAG & GraphRAG FLOW

13.1 Standard RAG Flow

User Query: "Why does Pump-21 overheat?"



Query Preprocessing

- Expand query: add synonyms, equipment context
- Detect entities: [PUMP-21, overheat, temperature]



Hybrid Retrieval

- Dense: embed query → ChromaDB search → top 20 chunks
- Sparse: BM25 keyword → PostgreSQL FTS → top 20 results
- Merge via Reciprocal Rank Fusion



Re-ranking

- Cross-encoder re-ranks merged results
- Filter by equipment_tag = "PUMP-21"
- Return top 8 most relevant chunks



Context Assembly

- Inject retrieved chunks into Claude prompt
- Add citation metadata (doc_id, page, chunk_index)



Claude Generation

- Reasons over context
- Generates grounded answer

- **Attaches citations for each claim**
- **Outputs confidence score**



Response to User

- **Answer text**
- **Citations with document name + page**
- **Confidence: 87%**
- **Related documents: [3 suggestions]**

13.2 GraphRAG Flow

Used when query needs relationship traversal (not just similarity search)

Query: "Show everything connected to Pump-21"



Entity Extraction

- **Identify: PUMP-21 (Equipment node)**



Graph Traversal (Neo4j)

```
MATCH (e:Equipment {tag:"PUMP-21"})-[*1..2]-(n)
```

```
RETURN n, labels(n), type(r)
```



Node Content Retrieval

- **For each connected Document node → fetch top chunks**
- **For each Incident node → fetch description**
- **For each Engineer node → fetch expertise**



Context Synthesis

- **Combine graph structure + document chunks**

- Build relationship-aware context



Claude Generation

- Reasons over connected context
- Response includes relationship context
- "According to OEM Manual (linked to PUMP-21), and corroborated by the June 2025 incident report..."

13.3 Hybrid RAG + GraphRAG (RCA Mode)

Query: "Root cause of Pump-21 bearing failure June 2025"



Graph: Get all nodes connected to PUMP-21 + June 2025 Incident



RAG: Search maintenance logs, manuals, similar incidents



Merge: Graph context + RAG chunks



Claude RCA Agent: Systematic investigation → structured report

14. API Design

REST APIs

Authentication

Assets

Documents

Search

Reports

Users

Example

POST /documents/upload

GET /machines

POST /chat

GET /reports

POST /workflow

Base URL: <https://api.factoryos.ai/v1>

Authentication

All endpoints require Bearer token in Authorization header.

Document APIs

POST	/documents/upload	Upload a document
GET	/documents/{id}	Get document metadata
GET	/documents/	List documents (with filters)
DELETE	/documents/{id}	Delete document

GET /documents/{id}/chunks Get chunks for a document

POST /documents/compare Compare two documents

POST /documents/upload

json

Request:

```
{
  "file": "<multipart>",
  "equipment_tag": "PUMP-21",
  "doc_type": "maintenance_log",
  "plant_id": "uuid"
}
```

Response:

```
{
  "document_id": "uuid",
  "status": "processing",
  "estimated_completion_seconds": 15,
  "entities_extracted": {
    "equipment_tags": ["PUMP-21"],
    "dates": ["2025-06-15"],
    "personnel": ["Ravi Kumar"]
  }
}
```

Query APIs

POST /query Natural language query

- POST /query/rca Root cause analysis
- POST /query/compare Document comparison
- GET /query/history User's query history

POST /query

json

Request:

```
{  
  
  "query": "Why did Pump-21 fail last month?",  
  
  "equipment_tag": "PUMP-21",  
  
  "filters": {  
  
    "date_from": "2025-05-01",  
  
    "doc_types": ["maintenance_log", "incident", "manual"]  
  
  },  
  
  "conversation_id": "uuid"  
}
```

Response:

```
{  
  
  "answer": "Based on the June 2025 maintenance log and OEM manual...",  
  
  "confidence_score": 0.87,  
  
  "citations": [  
  
    {  
  
      "document_id": "uuid",  
  
      "document_name": "PUMP-21-Maintenance-June2025.pdf",  
  
      "page": 3,  
  
      "excerpt": "Bearing temperature exceeded 95°C...",  
  
    },  
  
  ],  
  
}
```

```
      "relevance_score": 0.94
    }
  ],
  "related_documents": ["uuid1", "uuid2"],
  "agent_used": "rca_agent",
  "conversation_id": "uuid",
  "response_time_ms": 2340
}
```

POST /query/rca

json

Request:

```
{
  "equipment_tag": "PUMP-21",
  "failure_description": "Bearing failure with overheating",
  "occurrence_date": "2025-06-15"
}
```

Response:

```
{
  "equipment": "PUMP-21",
  "failure": "Bearing failure with overheating",
  "timeline": [...],
  "probable_causes": [
    {
      "cause": "Lubrication failure",
      "probability": 0.78,
```

```
    "evidence": [...]  
  }  
],  
  "recommendations": [...],  
  "preventive_measures": [...],  
  "confidence_score": 0.82,  
  "report_id": "uuid"  
}
```

Compliance APIs

- GET /compliance/dashboard Compliance overview
 - GET /compliance/gaps List compliance gaps
 - POST /compliance/audit-package Generate audit evidence package
 - PUT /compliance/items/{id} Update compliance item status
-

Shift Intelligence APIs

- POST /shifts/handover Submit shift notes for AI processing
- GET /shifts/handover/{id} Get processed handover
- GET /shifts/ List all handovers

POST /shifts/handover

json

Request:

```
{  
  "plant_id": "uuid",  
  "shift_type": "night",
```

```
"shift_date": "2025-07-01",

"outgoing_engineer_id": "uuid",

"raw_notes": "Pump-21 showing vibration since 2am. Compressor-3 oil changed. Line B shutdown for valve replacement. Boiler-12 running at 85% capacity due to feedwater issue."

}
```

Response:

```
{

"handover_id": "uuid",

"structured_handover": {

"equipment_status": [

{"tag": "PUMP-21", "status": "attention_needed", "note": "Vibration observed since 02:00"},

{"tag": "COMPRESSOR-3", "status": "normal", "note": "Oil changed, monitoring"},

{"tag": "BOILER-12", "status": "reduced_capacity", "note": "85% capacity, feedwater issue"}

],

"critical_alerts": [

"PUMP-21 vibration — immediate inspection recommended before morning operations"

],

"pending_actions": [

"Line B valve replacement — in progress, ETA 06:00",

"Boiler-12 feedwater issue — requires feedwater treatment check"

],

"completed_work": [

"Compressor-3 scheduled oil change completed"

]

}

}
```

Knowledge Graph APIs

- GET /graph/equipment/{tag}** **Get equipment graph context**
 - GET /graph/visualize** **Graph data for visualization**
 - POST /graph/query** **Custom Cypher-like query**
 - GET /graph/similar-incidents** **Find similar incidents**
-

Incident APIs

- POST /incidents/** **Report new incident**
 - GET /incidents/{id}** **Get incident details**
 - POST /incidents/{id}/learn** **Trigger learning loop**
 - GET /incidents/patterns** **Get recurring patterns**
-

15. Folder Structure

factoryos-ai/

frontend/

backend/

agents/

rag/

knowledge_graph/

database/

api/

docs/

tests/

docker/

Frontend (React + TypeScript)

factoryos-frontend/

├── public/

| ├── favicon.ico

| └── assets/

├── src/

| ├── components/

| | ├── ui/ # shadcn/ui base components

| | | ├── Button.tsx

| | | └── Card.tsx

- └─ **Badge.tsx**
 - └─ **Input.tsx**
 - └─ ...
 - └─ **layout/**
 - └─ **Sidebar.tsx**
 - └─ **Header.tsx**
 - └─ **PageWrapper.tsx**
 - └─ **MobileNav.tsx**
 - └─ **industrial-gpt/**
 - └─ **ChatInterface.tsx**
 - └─ **MessageBubble.tsx**
 - └─ **CitationCard.tsx**
 - └─ **ConfidenceBadge.tsx**
 - └─ **QuerySuggestions.tsx**
 - └─ **documents/**
 - └─ **DocumentUpload.tsx**
 - └─ **DocumentList.tsx**
 - └─ **DocumentViewer.tsx**
 - └─ **DocumentCard.tsx**
 - └─ **StaleDocumentAlert.tsx**
 - └─ **knowledge-graph/**
 - └─ **GraphViewer.tsx**
 - └─ **NodeDetail.tsx**
 - └─ **GraphFilters.tsx**
 - └─ **rca/**
 - └─ **RCAForm.tsx**

- └─ RCAReport.tsx
 - └─ EvidenceTimeline.tsx
 - └─ CauseCard.tsx
 - └─ compliance/
 - └─ ComplianceDashboard.tsx
 - └─ GapList.tsx
 - └─ AuditPackageGenerator.tsx
 - └─ shift/
 - └─ ShiftHandoverForm.tsx
 - └─ HandoverReport.tsx
 - └─ CriticalAlertBanner.tsx
 - └─ incidents/
 - └─ IncidentForm.tsx
 - └─ LessonsLearned.tsx
 - └─ PatternAlert.tsx
 - └─ dashboard/
 - └─ ExecutiveDashboard.tsx
 - └─ EquipmentHealthGrid.tsx
 - └─ KPICard.tsx
 - └─ AlertFeed.tsx
 - └─ pages/
 - └─ Home.tsx
 - └─ IndustrialGPT.tsx
 - └─ Documents.tsx
 - └─ KnowledgeGraph.tsx
 - └─ RCA.tsx

- └─ **Compliance.tsx**
 - └─ **ShiftHandover.tsx**
 - └─ **Incidents.tsx**
 - └─ **PredictiveMaintenance.tsx**
 - └─ **Equipment.tsx**
 - └─ **Dashboard.tsx**
 - └─ **hooks/**
 - └─ **useQuery.ts**
 - └─ **useDocuments.ts**
 - └─ **useGraph.ts**
 - └─ **useShift.ts**
 - └─ **useCompliance.ts**
 - └─ **services/**
 - └─ **api.ts** **# Base API client**
 - └─ **queryService.ts**
 - └─ **documentService.ts**
 - └─ **complianceService.ts**
 - └─ **shiftService.ts**
 - └─ **store/**
 - └─ **appStore.ts** **# Zustand global state**
 - └─ **queryStore.ts**
 - └─ **documentStore.ts**
 - └─ **types/**
 - └─ **document.types.ts**
 - └─ **query.types.ts**
 - └─ **equipment.types.ts**

```
| | └─ api.types.ts
| └─ utils/
| | └─ formatters.ts
| | └─ citations.ts
| | └─ constants.ts
| └─ App.tsx
└─ main.tsx
└─ package.json
└─ tailwind.config.js
```

Backend (FastAPI + Python)

factoryos-backend/

```
└─ app/
|   └─ main.py           # FastAPI app entry
|   └─ config.py         # Environment config
|   └─ database/
|       └─ postgres.py   # PostgreSQL connection
|       └─ neo4j.py       # Neo4j connection
|       └─ chroma.py      # ChromaDB connection
|       └─ redis.py       # Redis connection
|   └─ models/
|       └─ document.py
|       └─ equipment.py
|       └─ incident.py
|       └─ shift.py
|       └─ user.py
└─ api/
```

- | | └─ routes/
- | | | └─ documents.py
- | | | └─ query.py
- | | | └─ compliance.py
- | | | └─ shifts.py
- | | | └─ incidents.py
- | | | └─ equipment.py
- | | | └─ graph.py
- | | └─ middleware/
- | | | └─ auth.py
- | | | └─ rate_limit.py
- | | | └─ audit_log.py
- | └─ agents/
- | | └─ orchestrator.py
- | | └─ document_intelligence.py
- | | └─ search_retrieval.py
- | | └─ industrial_gpt.py
- | | └─ rca_agent.py
- | | └─ predictive_maintenance.py
- | | └─ compliance_agent.py
- | | └─ knowledge_graph_agent.py
- | | └─ shift_intelligence.py
- | | └─ incident_learning.py
- | | └─ knowledge_decay.py
- | | └─ vision_ai.py
- | | └─ workflow_agent.py

- | | | └─ report_generator.py
- | | | └─ memory_agent.py
- | | | └─ executive_intelligence.py
- | | └─ supply_chain_agent.py
- | └─ ingestion/
 - | | └─ pipeline.py # Main ingestion orchestrator
 - | | └─ ocr.py # PaddleOCR wrapper
 - | | └─ pdf_parser.py
 - | | └─ chunker.py
 - | | └─ embedder.py
 - | | └─ entity_extractor.py
 - | | └─ classifier.py
- | └─ rag/
 - | | └─ hybrid_search.py
 - | | └─ reranker.py
 - | | └─ graph_rag.py
 - | | └─ context_builder.py
- | └─ graph/
 - | | └─ schema.py
 - | | └─ queries.py
 - | | └─ builder.py
- | └─ services/
 - | | └─ claude_service.py # Claude API wrapper
 - | | └─ storage_service.py # MinIO wrapper
 - | | └─ notification_service.py
- | └─ tasks/

- | |— celery_app.py
- | |— ingestion_tasks.py
- | |— decay_check_tasks.py
- |— tests/
 - | |— test_agents/
 - | |— test_api/
 - | |— test_rag/
 - | |— test_ingestion/
- |— docker/
 - | |— Dockerfile.backend
 - | |— Dockerfile.frontend
- |— docker-compose.yml
- |— requirements.txt
- |— .env.example

16. UI/UX Sitemap

Hierarchy

Login



Dashboard



Documents



Chat



Knowledge Graph



Digital Twin



Reports



Settings

FactoryOS AI

└─ / (Home — Executive Dashboard)

| └─ Plant health overview

| └─ Active alerts feed

| └─ KPI cards (uptime, compliance score, open incidents)

| └─ Quick actions



└─ /gpt (Industrial GPT)

- | |— Chat interface
- | |— Query suggestions
- | |— Citation panel (right)
- | |— Related documents panel
- | |— Conversation history
- |
- |— /documents (Document Intelligence)
- | |— Upload zone
- | |— Document library (grid/list)
- | |— Filter by type/equipment/date
- | |— Stale document alerts
- | |— /documents/:id (Document viewer + metadata)
- |
- |— /graph (Knowledge Graph)
- | |— Interactive graph visualization
- | |— Node search
- | |— Filter by entity type
- | |— Node detail panel
- |
- |— /rca (Root Cause Analysis)
- | |— Failure input form
- | |— Investigation progress
- | |— /rca/:id (Full RCA report)
 - | |— Evidence timeline
 - | |— Cause probability chart
 - | |— Recommendation list

- └─ Download report

- └─ /compliance (Compliance Intelligence)

- └─ Compliance score dashboard

- └─ Gap list by framework

- └─ Equipment compliance status

- └─ Audit package generator

- └─ /shifts (Shift Intelligence)

- └─ Submit handover form

- └─ Handover history

- └─ /shifts/:id (Structured handover view)

- └─ /incidents (Incident Learning)

- └─ Report incident form

- └─ Incident list + filters

- └─ Pattern detection dashboard

- └─ /incidents/:id (Incident detail + lessons)

- └─ /equipment (Equipment Registry)

- └─ Equipment list with health scores

- └─ Filter by type/status/plant

- └─ /equipment/:tag (Equipment detail)

- └─ All linked documents

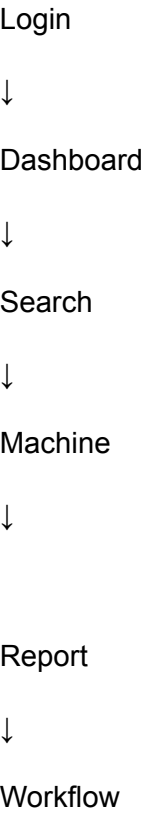
- └─ Maintenance history

- └─ Health trend chart

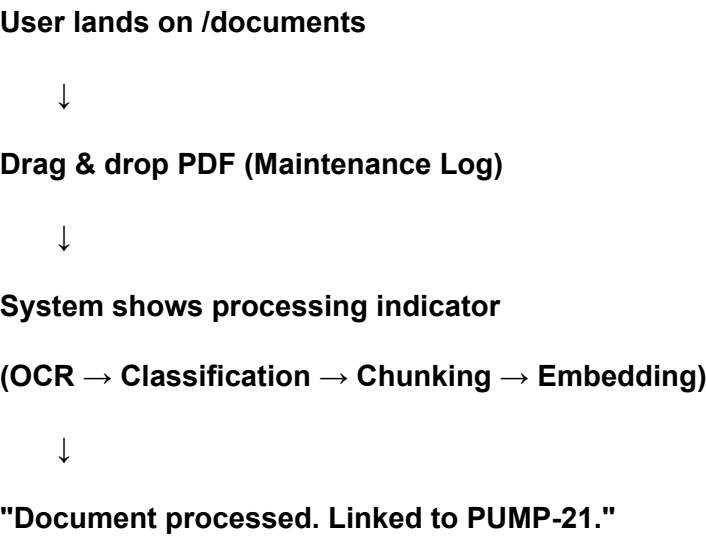
- | | — **Connected graph view**
 - | | — **Related incidents**
 - |
 - | — **/predictive (Predictive Maintenance)**
 - | | — **Equipment health grid**
 - | | — **RUL forecasts**
 - | | — **Maintenance schedule calendar**
 - |
 - | — **/settings**
 - | | — **Plant configuration**
 - | | — **User management**
 - | | — **Compliance framework setup**
 - | | — **Knowledge decay thresholds**
-

17. Screen Flow

User navigation.



Flow 1 — Document Upload & Query





User navigates to /gpt



Types: "What does the June maintenance log say about Pump-21?"



System shows typing indicator (2–4 seconds)



Response appears with:

- Answer paragraph
- 3 citation cards (document name + page + excerpt)
- Confidence: 91%
- Related: [2 suggested documents]



User clicks citation → Document viewer opens at that page

Flow 2 — RCA Investigation

User navigates to /rca



Form: Equipment Tag = "PUMP-21"

Failure = "Bearing failure with high temperature"

Date = 2025-06-15



Clicks "Investigate"



Progress screen:

- ✓ Gathering maintenance history
- ✓ Searching relevant manuals

- ✓ Finding similar incidents
- ✓ Analyzing sensor anomalies
- ✓ Building timeline
- ... Generating report



RCA Report page:

- Timeline (visual)
- 3 probable causes with probability bars
- Evidence for each cause (with citations)
- 5 recommended actions
- 3 preventive measures
- Download as PDF button

Flow 3 — Shift Handover

Engineer opens /shifts at end of shift



Types or pastes raw shift notes



Clicks "Process Handover"



AI structures notes in 3 seconds



Review screen shows:

-  Critical Alerts (PUMP-21 vibration)
-  Pending Actions (Line B valve)
-  Completed Work (Compressor-3 oil)

Equipment Status Grid



Engineer confirms and submits



Incoming engineer sees structured brief on login



18. Component Library

- Buttons
- Cards
- Tables
- Charts
- Dialogs
- AI Chat
- Graph Viewer
- Timeline
- Sidebar
- Navbar
- Forms

Design Tokens

CSS

```
:root {  
  
  /* Industrial Color Palette */  
  
  --color-primary: #0052CC;    /* Industrial Blue */  
  
  --color-danger: #DE350B;     /* Alert Red */  
  
  --color-warning: #FF991F;    /* Warning Amber */  
  
  --color-success: #00875A;    /* Safe Green */  
  
  --color-neutral-900: #091E42; /* Dark text */  
  
  --color-neutral-100: #F4F5F7; /* Light background */  
  
  /* Typography */
```

```
--font-primary: 'Inter', sans-serif;

--font-mono: 'JetBrains Mono', monospace;

/* Spacing */

--space-xs: 4px;

--space-sm: 8px;

--space-md: 16px;

--space-lg: 24px;

--space-xl: 40px;

}
```

Core Components

| Component | Purpose | Props |
|-------------------|-----------------------------|---|
| <EquipmentBadge> | Shows equipment tag | tag, status, healthScore |
| <CitationCard> | Displays AI citation | documentName, page, excerpt, relevanceScore |
| <ConfidenceMeter> | Visual confidence indicator | score (0-1) |
| <RCATimeline> | Visual evidence timeline | events[] |
| <ComplianceMeter> | Compliance percentage ring | score, framework |

| | | |
|------------------------------------|-------------------------------------|--|
| <AlertBanner> | Critical alert display | severity, message, equipmentTag |
| <DocumentCard> | Document in library | name, type, date, isStale |
| <HealthScore> | Equipment health gauge | score, trend |
| <AgentProgress> | Shows agent thinking steps | steps[], currentStep |
| <KnowledgeDecayAlert> | Stale document warning | documentName, age, riskLevel |
| <ShiftStatusGrid> | Equipment status in handover | equipment[] |
| <GraphNode> | Knowledge graph node | type, label, connectionCount |

19. User Journey

Example

Engineer



Login



Open Pump



Ask AI



See Root Cause



Generate Report



Schedule Maintenance

Output: ~10 pages

Complete Journey — Field Maintenance Engineer

Monday Morning — Receiving Shift Handover

1. Ravi logs into FactoryOS on tablet
2. Dashboard shows:  1 Critical Alert (PUMP-21 vibration since 02:00)
3. He opens the night shift handover report — sees structured AI-processed brief
4. Clicks on PUMP-21 → Equipment page opens showing full context

Investigating PUMP-21 5. He clicks "Investigate" → RCA form pre-filled with PUMP-21 6. Adds: "High vibration, bearing area hot to touch" 7. RCA agent runs → 4 minutes later: "92% probability: early bearing wear. Similar to June 2025 incident. Recommended: replace bearing SKF-6205-2RS. Part available in store (12 units)." 8. Ravi checks bearing in store → confirmed available 9. He schedules maintenance for 10am

Documentation 10. After maintenance, Ravi uploads photo of replaced bearing 11. Vision AI confirms: "Old bearing shows pitting wear consistent with lubrication starvation" 12. Ravi adds maintenance note: "Replaced bearing. Root cause: insufficient lubrication" 13. System automatically: creates maintenance record, links to PUMP-21, updates health score to 95, alerts Incident Learning Agent

Incident Learning Loop Triggers 14. Incident Learning Agent detects: "Same failure mode as June 2025 incident. Sibling equipment PUMP-22 and PUMP-23 use same bearing type." 15. Notifications sent to engineers responsible for PUMP-22 and PUMP-23 16. System suggests SOP update: "Add monthly lubrication check to MAINT-PM-PUMP-001"

End of Shift 17. Ravi opens Shift Handover form 18. Types: "PUMP-21 bearing replaced. Running normal. PUMP-22 and 23 lubrication check recommended per Incident Learning alert." 19. AI structures it in 2 seconds 20. Incoming shift engineer receives clean handover

20. DEVELOPMENT ROADMAP

Phase 1

Knowledge

Phase 2

Visibility

Phase 3

Prediction

Phase 4

Automation

Phase 5

Optimization

Phase 6

Autonomous Factory

Phase 1 — Foundation (Week 1–2)

Goal: Core infrastructure + document ingestion working

- Setup React + TypeScript frontend skeleton
- Setup FastAPI backend
- PostgreSQL + ChromaDB + MinIO configured
- Document upload API working
- PDF text extraction + chunking + embedding pipeline
- Basic Claude API integration (ask a question, get answer)
- Simple UI: upload document + ask question

Milestone: Can upload a PDF and ask questions about it

Phase 2 — Industrial GPT + RAG (Week 3–4)

Goal: Production-quality Q&A with citations

- Hybrid search (dense + sparse) implemented
- Citation attachment to responses
- Confidence score generation
- Multi-turn conversation (conversation history)
- Equipment-scoped search (filter by tag)
- Document comparison feature
- Frontend chat interface with citation cards

Milestone: Full Industrial GPT working — ask any question, get cited answer

Phase 3 — Knowledge Graph + RCA (Week 5–6)

Goal: Graph-connected intelligence + RCA reports

- Neo4j setup + schema deployed
- Entity extraction from documents → graph nodes
- Equipment-Document-Incident relationships
- Graph visualization component
- RCA Agent implemented
- RCA API endpoint
- RCA UI (form + report + timeline)
- GraphRAG queries working

Milestone: Type a failure description → get full RCA report

Phase 4 — Compliance + Shift Intelligence (Week 7–8)

Goal: Safety and handover workflows live

- OISD compliance checklist loaded
- Compliance gap detection agent
- Compliance dashboard UI
- Shift handover API + agent
- Shift handover UI (form + structured output)
- Knowledge Decay Detection (scheduled job)
- Stale document alerts in UI

Milestone: Compliance score visible + shift handover working

Phase 5 — Incident Learning + Predictive (Week 9–10)

Goal: Self-improving safety intelligence

- Incident form + reporting API
- Incident Learning Agent
- Similar incident detection
- Sibling equipment alerts
- Predictive maintenance (simulated sensor data)
- Equipment health score dashboard
- Vision AI (image upload + defect detection)

Milestone: Upload incident report → system learns + alerts related equipment

Phase 6 — Polish + Hackathon Demo (Week 11–12)

Goal: Demo-ready, production-quality

- Executive dashboard with KPIs
 - Report download (PDF generation)
 - Mobile responsive UI
 - Demo data loaded (oil & gas scenario)
 - Architecture diagram finalized
 - Presentation deck
 - Demo video recorded
 - Deployment on Railway/Render
-
-

21. SECURITY ARCHITECTURE

GitHub

↓

CI/CD

↓

Docker

↓

AWS

↓

Kubernetes (Future)

↓

Monitoring

↓

Logs

Output: ~15 pages

Authentication

JWT

RBAC

Encryption

Audit Logs

API Security

Secrets Management

Human Approval

Explainable AI

Output: ~15 pages

21.1 Authentication & Authorization

User Login



JWT Token issued (24hr expiry)



Every API request → Token validated



RBAC check:

- admin → full access
- manager → read all + approve workflows
- engineer → read + write for own plant
- safety → read all + write compliance
- viewer → read only



Document-level permissions:

- Each document has allowed_roles[]
- Confidential documents → explicit user whitelist

21.2 Data Security

| Layer | Security Measure |
|-------|------------------|
|-------|------------------|

| | |
|--------------------------|---|
| Data at rest | AES-256 encryption (PostgreSQL + MinIO) |
| Data in transit | TLS 1.3 (all API calls) |
| Vector embeddings | No PII stored in embeddings |
| API keys | Stored in environment variables, never in code |
| Claude API | API key rotated monthly |
| Passwords | bcrypt hashed (cost factor 12) |
| Audit log | Immutable audit trail for all AI queries |

21.3 Audit Trail

Every AI interaction is logged:

- User ID + timestamp
- Query text
- Agent used
- Citations returned
- Confidence score
- Response time

This audit trail is immutable and tamper-evident, critical for compliance use cases.

21.4 Data Isolation

- Each plant = separate ChromaDB collection namespace
- Each plant = separate Neo4j subgraph
- Cross-plant data sharing requires explicit admin permission
- Multi-tenant: one deployment can serve multiple plants in isolation

22. DEPLOYMENT ARCHITECTURE

MVP Deployment (Hackathon)

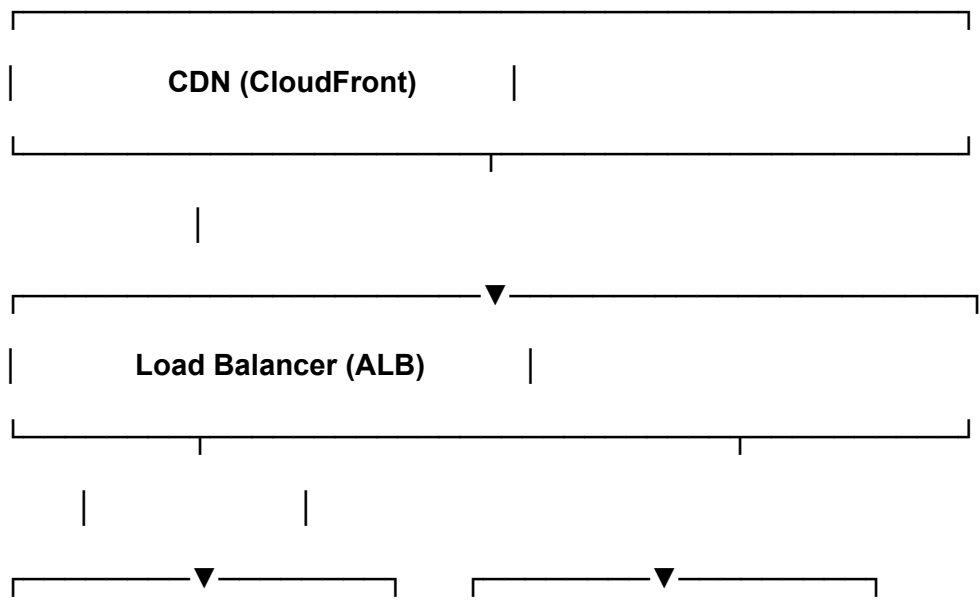
Railway / Render

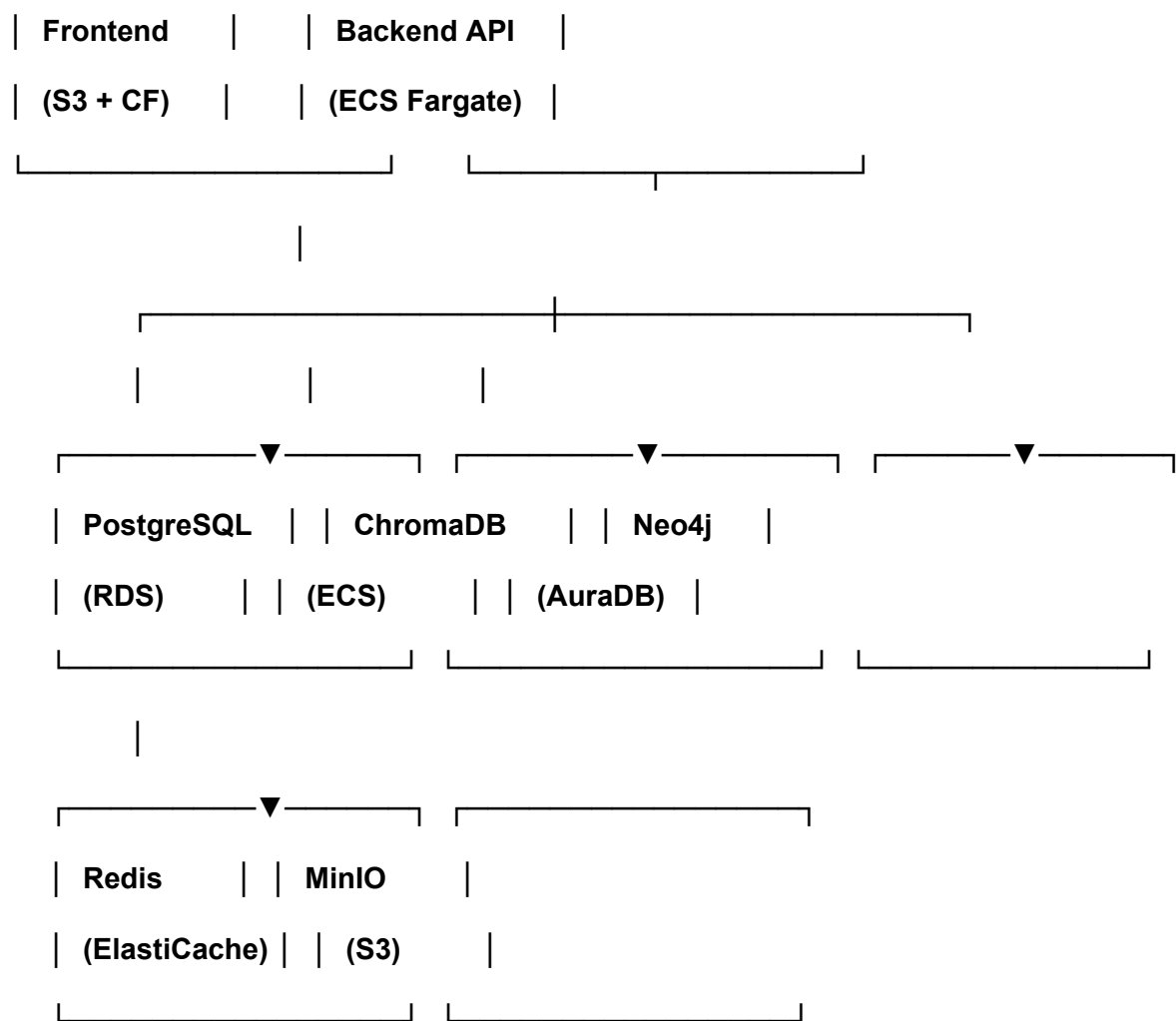
- factoryos-frontend (React static build)
- factoryos-backend (FastAPI)
- postgresql (managed)
- redis (managed)
- chromadb (containerized)

External:

- Claude API (Anthropic)
- Neo4j Aura (free tier)
- MinIO (object storage)

Production Deployment





docker-compose.yml (Local Dev)

yaml

version: '3.8'

services:

frontend:

build: ./factoryos-frontend

ports:

- "3000:3000"

environment:

- VITE_API_URL=http://localhost:8000

backend:

build: ./factoryos-backend

ports:

- "8000:8000"

environment:

- DATABASE_URL=postgresql://postgres:password@postgres:5432/factoryos
- REDIS_URL=redis://redis:6379
- NEO4J_URL=bolt://neo4j:7687
- ANTHROPIC_API_KEY=\${ANTHROPIC_API_KEY}

depends_on:

- postgres
- redis
- chromadb
- neo4j

postgres:

image: postgres:15

environment:

- POSTGRES_DB: factoryos
- POSTGRES_PASSWORD: password

volumes:

- postgres_data:/var/lib/postgresql/data

chromadb:

image: chromadb/chroma:latest

ports:

- "8001:8000"

volumes:

- chroma_data:/chroma/chroma

neo4j:

image: neo4j:5

ports:

- "7474:7474"

- "7687:7687"

environment:

NEO4J_AUTH: neo4j/password

volumes:

- neo4j_data:/data

redis:

image: redis:7-alpine

ports:

- "6379:6379"

celery_worker:

build: ./factoryos-backend

command: celery -A app.tasks.celery_app worker --loglevel=info

depends_on:

- redis

- postgres

volumes:

postgres_data:

chroma_data:

neo4j_data:

23. TESTING STRATEGY

- Unit Tests
- Integration Tests
- API Tests
- UI Tests
- AI Evaluation
- Load Testing
- Security Testing
- User Acceptance Testing

Output: ~15 pages

23.1 Unit Tests

| Component | Test Focus |
|-----------|------------|
|-----------|------------|

| | |
|---------------------|--|
| Document chunker | Chunk size, overlap, boundary handling |
| Entity extractor | Equipment tag detection accuracy |
| Embedding generator | Dimension consistency |
| RCA Agent | Evidence gathering completeness |
| Compliance Agent | Gap detection accuracy |
| Shift Agent | Structured output format correctness |

23.2 Integration Tests

| Flow | Test |
|-------------------------|---------------------------------|
| Upload → Ingest → Query | End-to-end document round-trip |
| RCA full flow | Failure input → complete report |
| Compliance check | Documents → gap list accuracy |
| Shift handover | Raw notes → structured output |

Knowledge Graph

Upload doc → graph nodes created

23.3 AI Quality Tests

| Metric | Method | Target |
|-----------------------|---------------------------|--------|
| Answer relevance | Ragas framework | > 0.85 |
| Citation accuracy | Manual golden set | > 90% |
| Hallucination rate | Factual consistency check | < 5% |
| RCA cause accuracy | Domain expert review | > 80% |
| Compliance gap recall | Against known gap set | > 95% |

23.4 Performance Tests

| Test | Tool | Target |
|---------------------|--------|-----------------|
| Query response time | Locust | p95 < 5 seconds |
| Document ingestion | Locust | 10MB PDF < 30s |

| | | |
|------------------|-----------------|--------------------------|
| Concurrent users | k6 | 50 users, no degradation |
| Graph query | Neo4j benchmark | < 2 seconds |

23.5 Test Data Strategy

- Create synthetic oil & gas facility dataset
 - 50+ documents (manuals, SOPs, incident reports, maintenance logs)
 - 10 equipment items with full history
 - Known compliance gaps for validation
 - Known RCA answers for 5 test failure scenarios
-

24. RISK ANALYSIS

- Risks
 - Poor OCR
 - Hallucinations
 - Security
 - Latency
 - Integration Complexity
- Mitigation
 - GraphRAG
 - Confidence Score
 - Human Review
 - Caching
 - Monitoring
- Output:** ~10 pages

Technical Risks

| Risk | Probability | Impact | Mitigation |
|----------------------------|-------------|--------|--|
| Claude API hallucination | Medium | High | Mandatory citations, confidence threshold, human review for critical decisions |
| OCR accuracy on bad scans | High | Medium | Pre-processing filters, manual upload fallback, confidence indicator |
| Neo4j performance at scale | Low | High | Query optimization, index design, result caching |

| | | | |
|---------------------------|--------|--------|---|
| Embedding drift over time | Low | Medium | Periodic re-embedding, versioned embedding models |
| Claude API rate limits | Medium | High | Request queuing, caching, fallback responses |

Business Risks

| Risk | Probability | Impact | Mitigation |
|---|-------------|--------|--|
| Industrial data sensitivity | High | High | On-premise deployment option, data isolation, encryption |
| Resistance from senior engineers | Medium | Medium | UX designed for non-tech users, Voice input, value demonstration |
| Integration complexity with legacy CMMS | High | Medium | File import-first approach, API integration as V2 |
| Compliance regulatory changes | Low | Medium | Modular compliance framework, admin-updatable rules |

Hackathon-Specific Risks

| Risk | Mitigation |
|--------------------------|--|
| Demo data quality | Prepare curated synthetic dataset in advance |
| API downtime during demo | Cache responses, prepare offline fallback |

Complex demo flow

Prepare 3 clear demo scenarios, rehearse 5+ times

Judges not understanding industrial context

Lead with problem statistics, not technology

25. FUTURE ROADMAP

6-Month Horizon

V2.0 — Enterprise Integrations

- SAP ERP integration (equipment master, work orders)
- CMMS API integration (IBM Maximo, SAP PM)
- SCADA live data connection
- Email ingestion (auto-process maintenance emails)
- WhatsApp bot for field engineers

V2.1 — Advanced AI

- Fine-tuned domain model on industrial corpora
- Multi-language support (Hindi, Tamil, Telugu)
- Voice interface for factory floor
- AI-generated SOPs from maintenance history

12-Month Horizon

V3.0 — Autonomous Operations

- Self-updating knowledge base (SOPs auto-revised by AI)
- Autonomous maintenance scheduling
- Predictive procurement (auto-generate POs)
- Digital twin integration with real IoT sensors

V3.1 — Multi-Plant Intelligence

- Cross-plant knowledge sharing (federated learning)
- Industry benchmarking (anonymous aggregates)
- Regional compliance frameworks (PESO, BIS, FSSAI)

24-Month Horizon

V4.0 — Industrial AI Marketplace

- Third-party agent plugins
- Industry-specific knowledge packs (Oil & Gas, Pharma, Steel)
- Vendor integration marketplace
- OEM portal (manufacturers publish machine AI agents)

Long-Term Vision

FactoryOS AI evolves from a knowledge platform into a self-learning Industrial Operating System — where every document, every maintenance activity, every incident, and every human interaction makes the factory smarter, safer, and more efficient.

The ultimate goal: an industrial organization where no knowledge is ever lost, no failure is ever repeated, and every engineer — from Day 1 — has access to the collective intelligence of the entire organization.

Document Version: 1.0 Author: Ayush Jain | Team CodeHack Project: FactoryOS AI Competition: AI for Industrial Knowledge Intelligence — Unified Asset & Operations Brain Status: Living Document — update as project evolves

Every Document. Every Machine. One AI Brain.

Future Roadmap

Future capabilities:

- Physics-Informed AI
- Autonomous Agents
- Edge AI
- Federated Learning
- Digital Expert Avatar
- ControlCode Copilot
- Sustainability Intelligence
- Industrial AR/VR
- Autonomous Plant Optimization